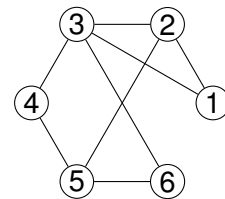


Laboratorio di Programmazione

Lezione 8: Esercizio Finale

Un *grafo* su n vertici è definito dall'insieme dei suoi *vertici*, $V = \{1, \dots, n\}$, e da un insieme A di *archi*, ognuno dei quali è una coppia di vertici (u, v) . L'insieme degli archi si denota E . Due vertici $u, v \in V$ del grafo sono *adiacenti* se formano un arco, cioè se $(u, v) \in E$. In pratica, i grafi vengono rappresentati visualmente connettendo i vertici adiacenti con un trattino. Qui sotto vedete un insieme E di 8 archi, ed il corrispondente grafo su $n = 6$ vertici.

$$E = \{(1, 2), (1, 3), (2, 3), (3, 4), (4, 5), (5, 6), (6, 3), (2, 5)\}$$



Un *triangolo* è un sottoinsieme di tre vertici $\{u, v, z\}$ mutuamente adiacenti, cioè tali che $(u, v), (u, z), (v, z) \in E$. Per esempio, nel grafo sopra $\{1, 2, 3\}$ è l'unico triangolo.

Scrivete un programma che legge da standard input l'intero n (sulla prima riga), seguito da E (un arco per riga). Il programma deve poi elencare tutti i triangoli del grafo. Il programma deve implementare le seguenti funzioni:

- `LeggiGrafo()` (n int, E []string) che ritorna il numero di vertici del grafo e la lista dei suoi archi (nota che gli archi sono ancora nel formato di input).
- `Arco(s string)` (u, v int) che converte un arco dal formato di input a coppia di interi, ordinati in modo che $u < v$.
- `Triangolo(u, v, z int, n int, E []string)` bool che verifica se $\{u, v, z\}$ è un triangolo.
- `Triangoli(n int, E []string)` int che restituisce il numero di triangoli nel grafo.

Nota: attenzione a non contare ogni triangolo più volte (es. potreste contare 6 volte il triangolo $\{1, 2, 3\}$, cioè una volta per ogni permutazione dei suoi vertici).

Esempio di utilizzo:

```
% go run triangolo.go
3
1 2
3 1
2 3
Trovati 1 triangoli:
1 2 3
```

```
% go run triangolo.go
4
1 2
2 3
4 1
2 4
4 3
Trovati 2 triangoli:
1 2 4
2 3 4
```

```
% go run triangolo.go
4
1 2
2 3
3 4
4 1
Trovati 0 triangoli.
```